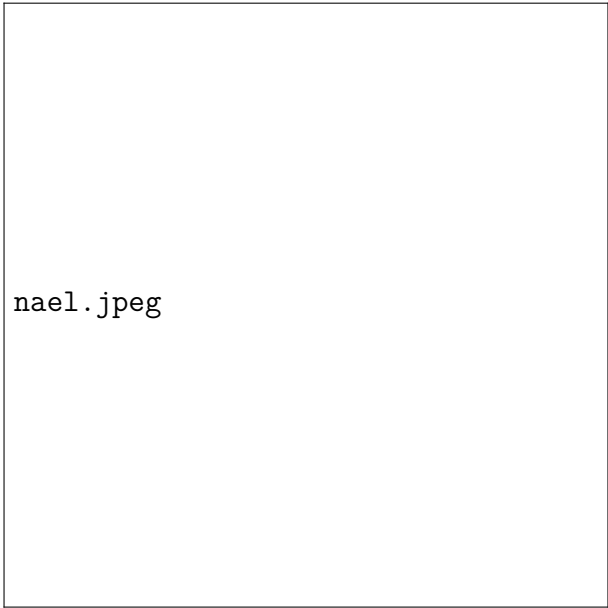


Laporan Tugas Besar

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 2: SYNTAX ANALYSIS

SEMESTER II / 2025–2026



nael.jpeg

Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021

Nicholas Wise Saragih Sumbayak - 13524037

Kurt Mikhael Purba - 13524065

Rava Khoman Tuah Saragih - 13524107

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

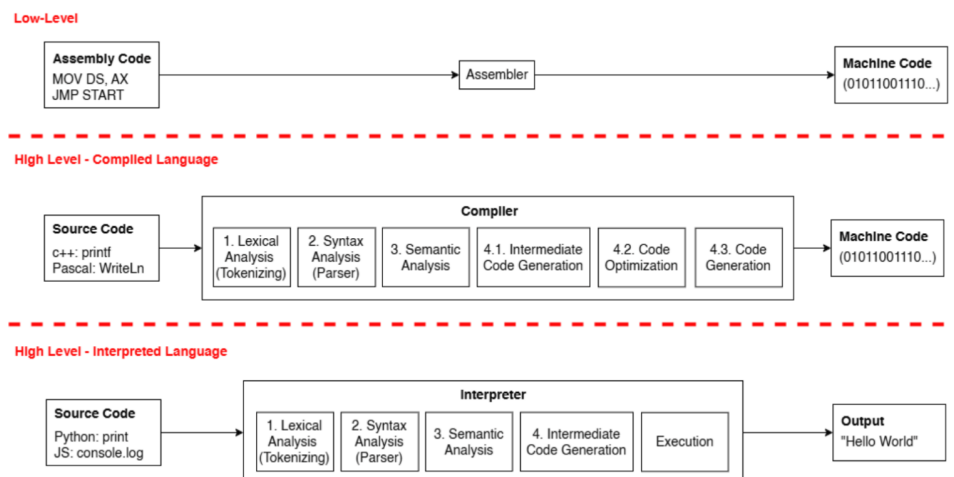
1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Rumusan Masalah	4
1.3	Gambaran Umum Sistem	4
1.4	Spesifikasi Umum	4
2	Dasar Teori	5
2.1	Pendefinisian Bahasa Pemrograman	5
2.1.1	Hierarki Bahasa Pemrograman	5
2.1.2	<i>Compiler</i> dan <i>Interpreter</i>	5
2.2	Teori Kompilator (<i>Compiler Theory</i>)	6
2.2.1	Analisis Leksikal (<i>Lexical Analysis</i>)	6
2.3	Teori Bahasa Formal dan Otomata	7
2.3.1	<i>Regular Expression</i>	7
2.3.2	<i>Finite Automata</i> (FA)	7
3	Perancangan <i>Lexical Analyzer</i>	8
3.1	Diagram Transisi DFA	8
3.2	Struktur Program <i>Lexical Analyzer</i>	12
3.2.1	Struktur Proyek	13
3.2.2	Klasifikasi Implementasi Alur DFA dalam Program	13
3.3	Token: Tipe dan Nilai	14
3.4	Hubungan Bahasa Reguler dan Pola Token	15
4	Implementasi	15
4.1	Pendekatan Modular pada DFA	15
4.1.1	Modul Identifier dan Kata Kunci (<i>q_identifyModule</i>)	15
4.1.2	Modul Bilangan (<i>q_number_module</i>)	16
4.1.3	Modul String dan Charcon	16
4.1.4	Modul Komentar	16
4.1.5	Modul Operator Dua Karakter	16
4.2	Mekanisme <i>Retract</i> dan <i>Lookahead</i>	17
4.3	Penanganan <i>Case-Insensitivity</i>	17
4.4	Struktur Kelas dan Tanggung Jawab	18
4.5	Struktur Direktori Proyek	18
5	Pengujian	19
5.1	Kasus Uji 1: Kata Kunci dalam Berbagai Huruf Besar/Kecil	19
5.2	Kasus Uji 2: Ambiguitas Bilangan Riil vs Titik	20
5.3	Kasus Uji 3: Charcon vs String	20
5.4	Kasus Uji 4: Komentar Bersarang dan Berkas Hanya Komentar	21
5.5	Kasus Uji 5: Konstruksi yang Tidak Ditutup	22
5.6	Kasus Uji 6: Operator == vs Karakter = Tunggal	23
5.7	Kasus Uji 7: Operator := vs : Diikuti Karakter Lain	24
5.8	Ringkasan Hasil Pengujian	25
6	Akhiran	26

6.1	Kesimpulan	26
6.2	Tantangan yang Dihadapi	26
References		27
Appendix		27
Pembagian Tugas		27

1 Pendahuluan

Pada tahap sebelumnya, telah berhasil dibangun sebuah *lexical analyzer* (lexer) yang mampu membaca kode sumber bahasa Arion dan mengubahnya menjadi daftar token. Setiap token merepresentasikan unit leksikal terkecil dari program, seperti kata kunci, identifier, konstanta, dan operator. Meskipun lexer telah berfungsi dengan baik dalam mengenali dan mengklasifikasikan token-token tersebut, lexer belum mampu memastikan apakah urutan token tersebut membentuk struktur program yang valid sesuai dengan aturan tata bahasa (*grammar*) bahasa Arion.

Proses transformasi kode sumber menjadi program yang dapat dijalankan melibatkan beberapa tahap berurutan. Setelah analisis leksikal, tahap berikutnya adalah analisis sintaktik (*syntax analysis*) atau yang sering disebut *parsing*. Pada tahap ini, parser menerima daftar token dari lexer dan memeriksa apakah urutan token tersebut sesuai dengan aturan grammar yang didefinisikan untuk bahasa Arion. Parser kemudian membangun sebuah struktur hierarkis yang disebut Parse Tree, yang merepresentasikan hubungan struktural antar-komponen program secara menyeluruh.



Gambar 1: Tahap-tahap kompilasi dalam compiler

Sumber: Asisten IF2224

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node dalam parse tree merepresentasikan simbol non-terminal atau terminal dari aturan produksi grammar. Berbeda dengan Abstract Syntax Tree (AST) yang menyederhanakan struktur dengan menghapus node-node informatif, Parse Tree memuat semua detail simbol seolah-olah seseorang menggambarkan semua langkah produksi secara lengkap. Parse Tree berguna untuk menunjukkan bagaimana token-token hasil lexical analysis membentuk struktur program yang sesuai dengan aturan grammar.

1.1 Latar Belakang

Proses kompilasi suatu bahasa pemrograman terdiri atas beberapa tahap yang saling berkaitan, dimulai dari analisis leksikal (*lexical analysis*), analisis sintaktik, analisis semantik, hingga pembangkitan kode (*code generation*). Setelah lexer berhasil mengonversi kode sumber menjadi deretan token, tahap analisis sintaktik menjadi penting untuk

memverifikasi apakah urutan token tersebut memenuhi aturan tata bahasa yang telah didefinisikan.

Pada tugas besar mata kuliah IF2224 Teori Bahasa Formal dan Otomata, mahasiswa diminta untuk membangun sebuah *compiler* sederhana untuk bahasa pemrograman Arion, yaitu bahasa bertipe Pascal. Milestone kedua ini berfokus pada pembangunan *syntax analyzer* (parser) yang mampu memeriksa urutan token dari lexer dan membangun Parse Tree sesuai dengan aturan grammar bahasa Arion. Parser yang dibangun menggunakan algoritma *Recursive Descent* yang mengimplementasikan setiap aturan produksi dalam grammar sebagai fungsi-fungsi rekursif.

1.2 Rumusan Masalah

Bagaimana merancang dan mengimplementasikan sebuah *syntax analyzer* berbasis *Recursive Descent Parser* yang mampu:

1. Membaca daftar token dari keluaran *lexical analyzer*.
2. Memeriksa apakah urutan token sesuai dengan aturan tata bahasa (*grammar*) bahasa Arion.
3. Mendeteksi dan melaporkan kesalahan sintaks (*syntax error*) dengan informasi lokasi yang akurat.
4. Membangun Parse Tree yang merepresentasikan struktur hierarkis program sumber.

1.3 Gambaran Umum Sistem

Sistem yang dibangun merupakan sebuah *syntax analyzer* untuk bahasa Arion yang diimplementasikan dalam bahasa C++20. Parser ini menerima daftar token dari lexer (berupa berkas teks yang berisi token-token), memproses setiap token menggunakan algoritma Recursive Descent, dan menghasilkan Parse Tree sebagai keluaran. Sistem terdiri atas beberapa komponen utama: Parser (mesin Recursive Descent), Node (struktur data node pada Parse Tree), TreeBuilder (konstruktor Parse Tree), dan ErrorHandler (pelaporan kesalahan sintaks).

1.4 Spesifikasi Umum

1. Menggunakan algoritma *Recursive Descent* dengan grammar yang dispesifikasikan dalam program.
2. Mendukung seluruh node Parse Tree sesuai spesifikasi: program, program-header, declaration-part, compound-statement, statement, expression, dan lainnya.
3. Melaporkan kesalahan sintaks dengan pesan yang informatif dan akurat.
4. Menghasilkan Parse Tree yang terformat dengan jelas, ditampilkan ke terminal dan disimpan ke dalam berkas teks.
5. Menerima masukan berupa berkas daftar token hasil dari lexer milestone pertama.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki kata kunci dan aturannya masing-masing serta makna dari kata-kata itu yang disebut sintaks(*syntax*) dan semantik (*semantics*). Input yang terdapat kesalahan pada tingkat sintaks berarti input tersebut tidak valid untuk bahasa pemrograman tersebut, sedangkan kesalahan pada tingkat semantik umumnya berbentuk kesalahan logika.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- ***Low-Level Language (Bahasa Tingkat Rendah)***: Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin (biner 0 dan 1). Contohnya adalah *Assembly*, yang memerlukan *Assembler* untuk menerjemahkan kode menjadi instruksi yang dapat dipahami CPU.
- ***High-Level Language (Bahasa Tingkat Tinggi)***: Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia karena menggunakan kosakata yang menyerupai bahasa natural. Semakin tinggi tingkat bahasa tersebut, semakin banyak tahapan pemrosesan yang diperlukan untuk mengubahnya menjadi kode mesin. Contoh bahasa tingkat tinggi antara lain Pascal, C++, Python, dan Java.

2.1.2 *Compiler* dan *Interpreter*

Untuk menjalankan kode sumber yang ditulis dalam bahasa tingkat tinggi, diperlukan sebuah perangkat lunak penerjemah yang mengubah kode tersebut menjadi format yang dapat dieksekusi oleh mesin. Secara umum, terdapat dua mekanisme utama untuk mengubah bahasa tingkat tinggi ke bahasa mesin:

1. ***Compiler (Kompilator)***: *Compiler* adalah program yang menerjemahkan seluruh *source code* secara utuh menjadi kode mesin (biasanya dalam bentuk *Object File*) sebelum program dijalankan. Proses kompilasi pada *compiler* umumnya terdiri dari empat tahapan utama: *Lexical Analysis*, *Syntax Analysis*, *Semantic Analysis*, dan *Intermediate Code Generation*. Karakteristik utama *compiler* adalah jika terdeteksi adanya kesalahan (*error*) di dalam *source code*, maka seluruh proses kompilasi akan gagal dan program tidak dapat dijalankan.
2. ***Interpreter***: Berbeda dengan *compiler*, *interpreter* memproses kode sumber baris demi baris secara langsung. *Interpreter* membaca, menerjemahkan, dan langsung mengeksekusi kode mesin tersebut baris demi baris. Hal ini memungkinkan program tetap berjalan normal hingga baris sebelum ditemukannya kesalahan; misalnya, jika terdapat *error* pada baris ke-10, baris 1 sampai 9 akan tetap berhasil dieksekusi. Meskipun mekanisme eksekusinya berbeda, *interpreter* sebenarnya memiliki tahapan pemrosesan internal yang serupa dengan *compiler*.

2.2 Teori Kompilator (*Compiler Theory*)

Proses kompilasi adalah urutan dari berbagai fase (*phases*) yang bekerja secara berurutan. Setiap fase mengambil input dari tahap sebelumnya, memprosesnya, dan memberikan output yang akan digunakan sebagai input untuk fase berikutnya. Secara umum, arsitektur sebuah kompilator terdiri dari fase-fase berikut:

1. **Lexical Analysis:** Fase pertama yang bekerja sebagai pemindai teks (*text scanner*) pada kode sumber. Penjelasan lebih detail mengenai fase ini dibahas pada sub-bab tersendiri di bawah.
2. **Syntax Analysis (Parsing):** Mengambil *token* yang dihasilkan oleh *lexical analysis* dan membangun pohon sintaksis (*parse tree*). Fase ini memeriksa kebenaran susunan *token* berdasarkan aturan tata bahasa (*grammar*) kode sumber.
3. **Semantic Analysis:** Memeriksa apakah pohon sintaksis yang dibangun mengikuti aturan semantik bahasa, seperti kecocokan operasi tipe data dan memastikan *identifier* telah dideklarasikan sebelum digunakan. Fase ini menghasilkan keluaran berupa pohon sintaksis yang dianotasi.
4. **Intermediate Code Generation:** Menghasilkan representasi kode perantara untuk mesin abstrak, yang posisinya menjembatani antara bahasa tingkat tinggi dan bahasa mesin.
5. **Code Optimization:** Mengoptimalkan kode perantara dengan menghapus baris kode yang tidak perlu dan mengatur ulang urutan instruksi. Tujuannya adalah untuk mempercepat eksekusi program tanpa membuang sumber daya (*CPU*, memori).
6. **Target Code Generation:** Merupakan fase sintesis akhir (*back-end*) yang menggunakan representasi kode perantara dan tabel simbol untuk menghasilkan program target atau bahasa mesin.

2.2.1 Analisis Leksikal (*Lexical Analysis*)

Sebagai fokus utama pada *milestone* ini, Analisis Leksikal adalah fase pertama dari sebuah kompilator. Fase ini memproses kode sumber yang ditulis dalam bentuk kalimat sintaksis dan memecahnya menjadi serangkaian unit bermakna yang disebut *token*.

Berdasarkan prinsip desain kompilator, mekanisme kerja dan tugas utama dari *lexical analyzer* meliputi:

- **Penyaringan Karakter:** *Lexical analyzer* membaca kode sumber sebagai aliran karakter (*character stream*) dan secara otomatis menghapus seluruh spasi kosong (*whitespace*) maupun komentar (*comments*) di dalam kode sumber.
- **Tokenisasi dan Pengenalan *Lexeme*:** Barisan karakter (*alphanumeric*) yang memiliki makna diidentifikasi sebagai *lexeme*. *Lexeme* ini kemudian dicocokkan dengan aturan baku atau pola (*pattern*)—yang umumnya didefinisikan melalui *regular expression*—untuk divalidasi sebagai sebuah *token* (seperti kata kunci, konstanta, *identifier*, atau operator).
- **Kerja Sama dengan *Syntax Analyzer (Parser)*:** *Lexical analyzer* bekerja sangat erat dengan *syntax analyzer*. Ia memeriksa keberadaan *token* yang legal dan

meneruskan data *token* tersebut kepada *syntax analyzer* hanya pada saat dibutuhkan atau diminta (*when it demands*).

- **Penanganan Kesalahan (*Error Handling*):** Apabila *lexical analyzer* memindai urutan karakter yang tidak valid atau tidak dapat diklasifikasikan ke dalam pola *token* apa pun, maka ia akan langsung menghasilkan pesan kesalahan (*error*).

2.3 Teori Bahasa Formal dan Otomata

2.3.1 *Regular Expression*

Regular Expression (RE) adalah sebuah notasi matematis formal yang digunakan untuk mendeskripsikan suatu bahasa reguler. Dalam kompilator, RE digunakan secara luas untuk mendefinisikan *pattern* (pola) dari *token* secara akurat. Dengan menggunakan operasi dasar seperti *Concatenation* (penyambungan), *Union* (penggabungan alternatif), dan *Kleene Closure* (perulangan nol kali atau lebih), pembuat bahasa dapat mendefinisikan aturan rumit, misalnya: "Sebuah identifier harus diawali dengan huruf, diikuti oleh kombinasi huruf atau angka dengan jumlah berapa pun."

2.3.2 *Finite Automata* (FA)

Finite Automata (FA) atau *Finite State Machine* (FSM) adalah model komputasi matematika abstrak yang digunakan untuk mengenali *Regular Expression*. Dalam konteks kompilator, FA berfungsi sebagai mesin pemeriksa yang menelusuri karakter masukan untuk memvalidasi apakah urutan karakter tersebut (*lexeme*) membentuk *token* yang sah sesuai aturan bahasa.

Secara formal, sebuah *Finite Automata* didefinisikan secara matematis sebagai 5-tuple $(Q, \Sigma, \delta, q_0, F)$, yang terdiri dari:

- Q : Himpunan hingga dari kedudukan atau *state*.
- Σ : Himpunan hingga dari simbol *input* atau alfabet (contoh: himpunan huruf, angka, dan simbol baca).
- δ : Fungsi transisi yang memetakan $Q \times \Sigma \rightarrow Q$ (menentukan *state* tujuan berdasarkan *state* saat ini dan simbol yang dibaca).
- q_0 : *State* awal (*initial state*), di mana $q_0 \in Q$. Mesin selalu memulai komputasi dari *state* ini.
- F : Himpunan *state* akhir atau penerima (*final/accepting state*), di mana $F \subseteq Q$.

Mesin FA membaca masukan berupa *string* karakter demi karakter secara sekuensial dari kiri ke kanan. Setiap kali membaca satu simbol, mesin akan berpindah dari *state* saat ini ke *state* berikutnya sesuai dengan aturan pada fungsi transisi (δ). Jika setelah seluruh karakter pada masukan selesai dibaca mesin berhenti pada salah satu *state* yang termasuk dalam himpunan F , maka masukan tersebut diterima (*accepted*) sebagai *token* yang valid. Sebaliknya, jika berhenti di luar F atau menemukan transisi yang tidak terdefinisi, maka terjadi *lexical error*.

Berdasarkan sifat penentuan fungsi transisinya, *Finite Automata* dibedakan menjadi dua jenis:

1. ***Nondeterministic Finite Automata (NFA)***: Suatu mesin di mana sebuah *state* dapat memiliki nol, satu, atau lebih jalur transisi untuk satu simbol *input* yang sama. NFA juga mengizinkan perpindahan *state* tanpa perlu membaca karakter *input* (disebut transisi epsilon atau ϵ -transition).
2. ***Deterministic Finite Automata (DFA)***: Suatu mesin di mana setiap *state* memiliki tepat satu jalur transisi secara mutlak untuk setiap simbol *input* di dalam Σ . Tidak ada ambiguitas jalur dan tidak ada transisi epsilon.

Dalam implementasi program *Lexical Analyzer* yang sebenarnya, model **DFA** adalah spesifikasi yang wajib digunakan. Hal ini karena sifat deterministiknya yang tidak ambigu memungkinkan aturan transisi dikonversi secara langsung menjadi algoritma terprogram menggunakan pendekatan *modular* dan penyeleksian kondisi, membaca satu demi satu karakter hingga menghasilkan klasifikasi *token* yang tepat.

3 Perancangan *Lexical Analyzer*

3.1 Diagram Transisi DFA

Berikut merupakan rancangan *Deterministic Finite Automata (DFA)* yang dikembangkan untuk mengenali seluruh pembentukan leksikal token pada bahasa pemrograman Arion.

Kategori Token	Current State	Input	Next State	Aksi / Keterangan
Identifier & Keyword	q0	[a-zA-Z]	q_ident	Huruf pertama
	q_ident	[a-zA-Z0-9]	q_ident	Loop huruf/angka
	q_ident	selain itu	accept	Cek keyword / identifier
Number	q0	[0-9]	q_number	Angka pertama
	q_number	[0-9]	q_number	Loop integer
	q_number	.	q_real_coma	Masuk desimal
	q_number	selain itu	q_intcon	Accept integer
	q_real_coma	[0-9]	q_real	Digit setelah titik
	q_real_coma	selain itu	q_error_1	Error format real
	q_real	[0-9]	q_real	Loop desimal
	q_real	selain itu	accept	Accept real
String & Char	q0	'	q_string	Mulai string
	q_string	selain ' dan EOF	q_string	Loop isi
	q_string	'	accept	Selesai string/char
	q_string	EOF	q_error_2	Error tidak ditutup
Operator Tunggal	q0	+	q_plus	Accept
	q0	-	q_minus	Accept
	q0	*	q_times	Accept
	q0	/	q_rdiv	Accept
	q0	[	q_lbrack	Accept
	q0	]	q_rbrack	Accept

Lanjut ke halaman berikutnya

Kategori Token	Current State	Input	Next State	Aksi / Keterangan
	q0	,	q_comma	Accept
	q0	;	q_semicolon	Accept
	q0	.	q_period	Accept
	q0	)	q_rparent	Accept
Operator Relasional	q0	<	q_<	Kombinasi
	q_<	=	q_leq	Accept (<=)
	q_<	>	q_neq	Accept (<>)
	q_<	selain itu	q_lss	Accept (<)
	q0	>	q_>	Kombinasi
	q_>	=	q_geq	Accept (>=)
	q_>	selain itu	q_gtr	Accept (>)
	q0	=	q_<=	Kombinasi
	q_<=	=	q_eq	Accept (==)
	q_<=	selain itu	q_error_3	Error
Assignment	q0	:	q_:	Kombinasi
	q_:	=	q_becomes	Accept (:=)
	q_:	selain itu	q_colon	Accept (:)
Komentar (* *)	q0	(	q_(<	Kombinasi
	q_(<	*	q_comment	Mulai komentar
	q_comment	selain *	q_comment	Loop isi
	q_comment	*)	accept	Selesai komentar
	q_comment	EOF	q_error_4	Error tidak ditutup
Komentar { }	q0	{	q_{	Mulai komentar
	q_{	}	accept	Selesai komentar
	q_{	selain }	q_{	Loop isi
	q_{	EOF	q_error_5	Error tidak ditutup
Whitespace	q0	spasi/tab/newline	q0	Diabaikan



Gambar 2: Diagram Transisi DFA untuk *Lexical Analyzer* Bahasa Arion (Part 1)



Gambar 3: Diagram Transisi DFA untuk *Lexical Analyzer* Bahasa Arion (Part 2)

Perancangan diagram transisi DFA ini merupakan representasi logis dari bagaimana *scanner* membaca karakter demi karakter (*character stream*) dan menentukan validitas sebuah *lexeme*. Berdasarkan diagram di atas, terdapat beberapa keputusan desain dan analisis perancangan *state* yang diterapkan:

1. Mekanisme *Look-ahead* pada Operator Relasional dan *Assignment*

Bahasa Arion memiliki operator yang terdiri dari satu karakter (seperti $+$, $-$, $=$) dan multi-karakter (seperti $\leq$, $\geq$, $<>$, $:=$). Pada DFA, hal ini ditangani melalui mekanisme *look-ahead*. Sebagai contoh, ketika mesin membaca karakter $<$, ia berpindah ke sebuah *intermediate state*. Dari sana, mesin akan "mengintip" karakter berikutnya:

- Jika karakter selanjutnya adalah $=$, mesin bertransisi ke state akhir untuk token `leq` ($\leq$).
- Jika karakter selanjutnya adalah $>$, mesin bertransisi ke state akhir untuk token `neq` ($<>$).
- Jika karakter selanjutnya adalah karakter di luar itu, mesin melakukan *retract* (mundurkan pembacaan) dan menerima input sebagai token `lss` ($<$).

2. Pemisahan State Bilangan Bulat (*intcon*) dan Riil (*realcon*)

Penelusuran angka diawali dengan pembacaan digit (0-9). Selama input berupa digit, mesin akan terus berputar pada *state* loop angka tersebut. Untuk mengakomodasi konstanta riil (*realcon*), DFA menyediakan jalur transisi khusus jika menemukan karakter titik (.). Mesin tidak akan langsung menerima titik tersebut sebagai operator *period*, melainkan menunggu datangnya digit lanjutan. Jika setelah titik dipastikan terdapat digit tambahan, aliran akan bergeser secara definitif ke rute final *realcon*. Jika tidak, mesin akan menangani titik tersebut sebagai leksikal terpisah.

3. Pengelolaan Literal dan Komentar (*Trap States*)

Literal teks dan karakter dipicu oleh tanda petik tunggal ('). DFA memiliki *state* khusus yang akan terus melahap semua karakter apa pun hingga menemukan tanda petik penutup. Perbedaan validasi panjang untuk *charcon* (panjang = 1) dan *string* (panjang > 1) didelegasikan pada proses operasional program pasca-DFA. Selain itu, DFA juga memetakan rute untuk membuang isi komentar. Pembacaan { atau kombinasi (diikuti * akan membawa mesin ke dalam semacam *trap state* yang terus menerus membaca karakter tanpa membentuk elemen baru, hingga akhirnya menemukan blok penutup } atau *).

4. Analisis Penggabungan State untuk Token *ident* dan *Keyword*

Salah satu keputusan desain yang paling krusial pada DFA ini adalah menyatukan penelusuran *Identifier* (*ident*) dan seluruh *Keyword* (seperti *program*, *var*, *begin*, dll) ke dalam satu jalur transisi gabungan yang sama (*q_ident/keyword*). Keputusan arsitektural ini diambil dengan pertimbangan berikut:

- **Kesamaan Leksikal dan Ekspresi Reguler:** Baik *keyword* maupun *identifier* dibentuk oleh pola ekspresi reguler yang ekuivalen sepenuhnya, yaitu $[a-z, A-Z][a-z, A-Z, 0-9_]*$. Keduanya wajib diawali oleh alfabet dan bebas diikuti oleh rentetan alfanumerik.
- **Mencegah Fenomena *State Explosion*:** Apabila ke-52 token *keyword* pada spesifikasi Arion dipetakan secara eksplisit huruf demi huruf membentuk sebuah graf berarah (seperti pada struktur data *Trie*), diagram transisi akan menjadi terlampau masif dan tidak proporsional. Hal ini juga memicu kerentanan ambiguitas saat membedakan awalan sebuah variabel (contoh: variabel *integerKu*) dengan *keyword* yang beririsan (*integer*).
- **Efisiensi Resolusi Pascakomputasi:** Melalui penggabungan jalur ini, tugas murni mesin DFA disederhanakan hanya untuk mengekstrak satu rentetan kata yang valid secara leksikal. Setelah **accepting state** tercapai, proses validasi dilimpahkan pada kode program. Algoritma akan melakukan pencarian (*lookup*) nilai *lexeme* tersebut pada Tabel *Keyword* terpusat (*Hash Map*). Jika ditemukan kesamaan, maka token secara spesifik diklasifikasikan sebagai *keyword* (misal: *varsy*). Sebaliknya jika hasil nihil, secara **default** token tersebut diresmikan sebagai *ident* reguler.

3.2 Struktur Program *Lexical Analyzer*

Tahapan pertama dalam *compiler* ini adalah *Lexical Analyzer* (Scanner) yang bertugas membaca *source code* karakter demi karakter dari *SourceBuffer* dan mengelompokkannya menjadi *Token*. Implementasi pada *Scanner.cpp* dan *Scanner.hpp* dibangun dengan

secara langsung mereplikasi logika kerja *Deterministic Finite Automaton* (DFA) yang telah dijabarkan sebelumnya.

3.2.1 Struktur Proyek

Struktur implementasi untuk *lexical analyzer* pada proyek ini berfokus pada folder `src/lexical` dengan pembagian berkas sebagai berikut:

- `SourceBuffer.hpp` & `.cpp`: modul pembaca berkas sumber yang menyediakan akses karakter per karakter secara aman, termasuk operasi utilitas *look-ahead*.
- `Scanner.hpp` & `.cpp`: deklarasi dan implementasi kelas `Scanner` yang menjadi jantung dari mesin *lexical analyzer*.
- `Token.hpp`: representasi unit objek token beserta atribut esensialnya (jenis terminal, nilai *lexeme*, dan metadata penunjuk baris).
- `TokenType.hpp`: definisi enumerasi dari seluruh simpul terminal (52 tipe token) yang dikenali dan dipersyaratkan oleh spesifikasi bahasa Arion.
- `Logger.hpp`: utilitas sistem pelaporan untuk memunculkan pesan peringatan diagnostik selama tahap kompilasi (*lexical errors*).

3.2.2 Klasifikasi Implementasi Alur DFA dalam Program

Siklus mesin direpresentasikan oleh perulangan `scanToken()`. Saat fungsi ini dipanggil, ia bertindak seolah-olah mesin me-reset kembali ke *Start State* (q_0). Percabangan logika implementasi didasarkan pada pembacaan karakter awal `advance()` dengan *switch-case* atau percabangan if, membentuk modul sebagai berikut:

- **Modul 1: Identifier & Keywords (`q_ident/keyword`)**
Jika evaluasi mengembalikan fungsi valid `isAlpha()`, siklus memanggil `Scanner::identifier()`. Dalam metode ini, program membaca karakter selanjutnya (*looping*) secara kontinu selama fungsi `isAlphaNumeric()` masih bernilai *true*. Saat bertemu karakter asing, iterasi diputus. Teks *lexeme* yang terakumulasi lantas diekstrak dan dicocokkan dengan kontainer tabel konstanta *keyword* (`unordered_map`). Kecocokan berbuah *Token Keyword* (`programsy`, `forsy`, dll); jika tidak, instansiasi mengembalikan *Token ident*.
- **Modul 2: Numbers - `intcon` & `realcon` (`q_number_int`, `q_real_number`)**
Menemukan awalan digit angka mendelegasikan alur ke `Scanner::number()`. Blok perulangan primitif mengekstrak deret murni bilangan bulat. Melalui pendekatan *look-ahead*, mesin kemudian menguji validitas karakter titik '.' di depannya. Konformitas *match* (titik diikuti digit) memicu program mengakumulasi komposit tipe nilai primitif bilangan riil (berujung *Token realcon*). Tanpa validasi tersebut, nilai default yang diteruskan secara presisi adalah *Token intcon*.
- **Modul 3: Strings & Chars (`q_string_char`)**
Pemicu petik tunggal '' melompat ke rutinitas `Scanner::stringLiteral()`. Secara sederhana program mengolah teks bebas di dalamnya hingga blok penutup kutip terkalibrasi. Selesai memindai, evaluasi dihitung: panjang tepat 1 karakter menyuntikkan kategori `charcon`, sedangkan lebih dari itu bermuara sebagai `string`.

Modul ini dibekali dengan interupsi *error handling* seandainya baris berakhir gan-tung (*Unterminated string*).

- **Modul 4: Operators & Delimiters**

Karakter tunggal terminal absolut (spt: +, *, ;, ,) memiliki *node accepting state* permanen dan langsung diregister menjadi simbol terminal. Tipe multi-operator memanfaatkan delegasi pergeseran via fungsi `match()`:

- Simbol : akan menguji metode `match('=')`. Retur presisi mencetak `becomes (:=)`. Ketiadaan retur berbuah terminal tunggal `colon (:=)`.
- Skema ekuivalen membentengi karakter < yang sanggup bercabang ke `leq (<=)`, `neq (< >)`, hingga *fallback* tunggal `lss (<)`. Serta pada > yang mencakup `geq (>=)` atau `gtr (>)`.

- **Modul 5: Penanganan Whitespace & Comments**

Whitespace murni layakanya spasi, tabulasi (`\t`), dan *carriage return* (`\r`) secara utuh digugurkan eksistensinya tanpa mencetak token (lompatan balik siklik pada DFA). Transisi *newline* (`\n`) dimanfaatkan pasif sebagai *tracker* eskalasi baris. Di ranah *comments*, insiden identifikasi pembuka blok (spt { atau (*)) mengak-tivasi pengurusan karakter *dummy loop* (*trap state*) ke dalam *buffer* kosong hingga mendeteksi entitas kutub pasangannya masing-masing.

Seluruh mekanisme program berpedoman pada manuver pergeseran *pointer* alami yang mencerminkan topologi transisi diagram. Kompilasi unit valid tersebut akan diteruskan secara konsisten menuju inisialisasi pada *constructor* statis dari kelas `Token`.

3.3 Token: Tipe dan Nilai

Sebuah *token* terdiri atas dua komponen utama: **tipe token** (*token type*) yang me-nunjukkan kategori leksikal, dan **nilai** (*value/lexeme*) yang menyimpan representasi teks aslinya. Sebagai contoh dalam bahasa Arion:

Leksem	Tipe Token	Keterangan
program	programsy	Kata kunci deklarasi program
myVar	ident	Identifier yang didefinisikan pengguna
42	intcon	Konstanta bilangan bulat
3.14	realcon	Konstanta bilangan riil
'a'	charcon	Konstanta karakter tunggal
'hello'	string	Literal string (lebih dari satu ka-rakter)
:=	becomes	Operator penugasan
==	eq1	Operator persamaan

Tabel 2: Contoh token dalam bahasa Arion

Bahasa Arion mendefinisikan total 52 jenis token yang mencakup literal, kata kunci, operator aritmetika, operator logika, operator perbandingan, tanda baca, serta token khusus untuk komentar dan akhir berkas.

3.4 Hubungan Bahasa Reguler dan Pola Token

Setiap jenis token dalam suatu bahasa pemrograman dapat dideskripsikan oleh sebuah ekspresi reguler (*regular expression*). Sebagai contoh:

- **Identifer:** `[a-zA-Z_][a-zA-Z0-9_]*` — dimulai oleh huruf atau *underscore*, diikuti oleh nol atau lebih karakter alfanumerik.
- **Integer:** `[0-9]+` — satu atau lebih digit.
- **Real:** `[0-9]+\.[0-9]+` — digit, titik, lalu digit lagi.
- **Operator dua karakter:** `:=, ==, <=, >=, <>` — urutan dua karakter tetap.

Berdasarkan teorema Kleene, setiap bahasa yang dapat dideskripsikan oleh ekspresi reguler merupakan bahasa reguler dan dapat dikenali oleh DFA. Oleh karena itu, pendekatan berbasis DFA memberikan jaminan teoritis bahwa semua pola token dapat dikenali secara efisien dan deterministik.

4 Implementasi

4.1 Pendekatan Modular pada DFA

DFA lexer Arion dirancang dengan pendekatan *modular*: state awal q_0 berfungsi sebagai *dispatcher* yang memeriksa karakter pertama dan mendelegasikan pemrosesan ke modul-modul khusus. Pendekatan ini dipilih karena:

1. Setiap modul menangani satu kategori token sehingga kompleksitas logika terisolasi.
2. Penambahan pola token baru cukup dilakukan pada modul yang relevan tanpa mengubah modul lain.
3. Memudahkan pengujian dan *debugging* per modul secara independen.

Berikut adalah modul-modul utama dalam DFA:

4.1.1 Modul Identifer dan Kata Kunci (*q_identifyModule*)

Modul ini dimasuki ketika karakter pertama adalah huruf atau *underscore*. Scanner terus membaca karakter alfanumerik hingga menemukan karakter non-alfanumerik, lalu melakukan *retract* (tidak mengonsumsi karakter terakhir). Leksem yang terkumpul dikonversi ke huruf kecil dan dicari dalam tabel kata kunci. Jika ditemukan, token kata kunci yang sesuai dihasilkan; jika tidak, token `ident` dihasilkan dengan leksem asli (mempertahankan huruf besar/kecil asli).

Listing 1: Penanganan identifer dan kata kunci

```

1 void Scanner::identifier() {
2     while (isAlphaNumeric(peek()))
3         advance();
4     string text = source.substring(start, current - start);
5     string lower = toLower(text);
6     auto it = keywords.find(lower);

```



```

7     if (it != keywords.end())
8         addToken(it->second);
9     else
10        addToken(TokenType::ident, Literal{text});
11 }

```

4.1.2 Modul Bilangan (*q_number_module*)

Modul ini dimasuki ketika karakter pertama adalah digit. Scanner membaca digit berturut-turut. Jika setelah titik (.) terdapat digit lagi, maka bilangan diklasifikasikan sebagai *realcon*; jika tidak, bilangan menjadi *intcon* dan titik diperlakukan sebagai token *period* terpisah.

Listing 2: Penanganan bilangan bulat dan riil

```

1 void Scanner::number() {
2     while (isdigit(static_cast<unsigned char>(peek())))
3         advance();
4     if (peek() == '.' &&
5         isdigit(static_cast<unsigned char>(peekNext()))) {
6         advance(); // konsumsi '.'
7         while (isdigit(static_cast<unsigned char>(peek())))
8             advance();
9         addToken(TokenType::realcon, /* nilai double */);
10    } else {
11        addToken(TokenType::intcon, /* nilai integer */);
12    }
13 }

```

4.1.3 Modul String dan Charcon

Modul ini dimasuki ketika karakter ' (kutip tunggal) ditemukan. Scanner membaca karakter hingga menemukan kutip penutup. Setelah isi literal diekstrak, panjangnya diperiksa: jika tepat satu karakter, token *charcon* dihasilkan; jika lebih (atau nol), token *string* dihasilkan. Jika kutip penutup tidak ditemukan hingga akhir berkas, kesalahan dilaporkan.

4.1.4 Modul Komentar

Dua gaya komentar didukung: komentar kurung kurawal { ... } dan komentar parenthesis-bintang (* ... *). Pada kedua kasus, seluruh isi komentar dikonsumsi tanpa menghasilkan token apa pun. Komentar **tidak bersarang**: pada gaya { }, karakter { di dalam komentar diabaikan dan komentar ditutup oleh } pertama yang ditemui.

4.1.5 Modul Operator Dua Karakter

Untuk operator yang terdiri atas dua karakter (: =, ==, < =, > =, < >), scanner menggunakan metode *match()* sebagai *lookahead*. Setelah membaca karakter pertama, *match()* memeriksa apakah karakter berikutnya melengkapi operator dua karakter. Jika ya, kedua karakter dikonsumsi; jika tidak, hanya karakter pertama yang menjadi token.

Listing 3: Contoh lookahead untuk operator :=

```
1 case ':':
2     if (match('='))
3         addToken(TokenType::becomes); // :=
4     else
5         addToken(TokenType::colon); // :
6     return;
```

4.2 Mekanisme *Retract* dan *Lookahead*

Scanner menggunakan dua pointer: **start** (awal leksem saat ini) dan **current** (posisi baca saat ini). Metode **advance()** mengonsumsi satu karakter dan memajukan **current**, sedangkan **peek()** dan **peekNext()** membaca karakter berikutnya **tanpa** memajukan pointer. Pola ini memungkinkan scanner melakukan keputusan secara deterministik tanpa perlu *backtracking* yang mahal.

4.3 Penanganan *Case-Insensitivity*

Kata kunci dalam bahasa Arion bersifat *case-insensitive*. Implementasinya dilakukan dengan mengonversi leksem ke huruf kecil menggunakan **toLowerCase()** sebelum pencarian di tabel kata kunci (**unordered_map**). Leksem asli tetap disimpan untuk keperluan pelaporan dan modul-modul berikutnya.

4.4 Struktur Kelas dan Tanggung Jawab

Kelas/Berkas	Tanggung Jawab
SourceBuffer	Membaca berkas kode sumber ke dalam buffer string. Menyediakan akses karakter per posisi (<code>at()</code>) dan ekstraksi substring (<code>substring()</code>). Mengisolasi operasi I/O dari logika DFA.
TokenType	Mendefinisikan <code>enum class TokenType</code> dengan 52 anggota dan fungsi <code>tokenTypeName()</code> untuk konversi ke string.
Token	Struktur data yang menyimpan tipe token, leksem, literal (berupa <code>variant<monostate, string, double></code>), dan nomor baris. Menyediakan fungsi <code>printToken()</code> untuk keluaran format standar.
Scanner	Mesin utama DFA. Mengelola pointer <code>start/current</code> , mengimplementasikan <code>scanToken()</code> dengan <i>switch-case</i> , serta mendelegasikan ke metode-metode modul.
Logger	Melaporkan kesalahan leksikal ke <code>stderr</code> dengan format <code>[line N] Error: pesan</code> .
main.cpp	Titik masuk program. Membaca nama berkas dari argumen, membuat objek <code>Scanner</code> , dan memanggil <code>printAllTokens()</code> .

Tabel 3: Pembagian tanggung jawab kelas

4.5 Struktur Direktori Proyek

Listing 4: Tata letak direktori proyek

```

1 HBB-Tubes-IF2224-2026/
2   src/
3       main.cpp           # Titik masuk program
4       lexical/
5           SourceBuffer.hpp/cpp  # Pembacaan berkas
6           TokenType.hpp        # Definisi tipe token
7           Token.hpp/cpp        # Struktur dan pencetakan
                                   token
8           Scanner.hpp/cpp      # Mesin DFA utama
9           Logger.hpp           # Pelaporan kesalahan
10  test/
11      input/                # Berkas-berkas uji masukan
12  doc/                      # Laporan dan dokumentasi
13  Makefile                  # Skrip kompilasi

```

Arsitektur modular yang digunakan pada lexer dirancang untuk memudahkan pengembangan pada milestone berikutnya. Daftar token yang dihasilkan oleh `Scanner` dapat

langsung dikonsumsi oleh *parser* (analisis sintaktik) tanpa modifikasi antarmuka. Struktur **Token** yang menyimpan informasi nomor baris memungkinkan pelaporan kesalahan yang akurat pada tahap analisis semantik. Selain itu, pemisahan **SourceBuffer** dari **Scanner** memungkinkan penggantian sumber masukan (misalnya dari string langsung untuk pengujian unit) tanpa mengubah logika DFA.

5 Pengujian

Pengujian dilakukan terhadap berbagai kasus masukan untuk memverifikasi kebenaran lexer. Setiap kasus uji dijalankan dengan perintah `make run FILE=<nama_berkas>`. Keluaran pada `stdout` (daftar token) dan `stderr` (pesan kesalahan) dibandingkan dengan hasil yang diharapkan.

5.1 Kasus Uji 1: Kata Kunci dalam Berbagai Huruf Besar/Kecil

Berkas: `edge_keywords_case.txt`

Masukan:

```
1 BEGIN While PROGRAM
2 End If THEN
3 REPEAT downto NOT
4 DIV Mod Or
```

Keluaran yang diharapkan: `beginsy, whilesy, programsy, endsy, ifsy, thensy, repeatsy, downtosy, notsy, idiv, imod, orsy` — total 12 token kata kunci.



testcase1.png

Gambar 4: Hasil pengujian kasus uji 1

Analisis: Seluruh kata kunci dikenali dengan benar meskipun ditulis dengan variasi huruf besar/kecil, membuktikan bahwa mekanisme `toLowerCase()` bekerja sesuai spesifikasi.

5.2 Kasus Uji 2: Ambiguitas Bilangan Riil vs Titik

Berkas: `edge_real_vs_period.txt`

Masukan:

```
1 end .
2 3.14
3 42.
4 100.0
5 5 .3
```

Keluaran yang diharapkan: `endsy, period, realcon (3.14), intcon (42), period, realcon (100), intcon (5), period, intcon (3)` — total 9 token.



testcase2.png

Gambar 5: Hasil pengujian kasus uji 2

Analisis: Scanner membedakan `realcon` dan `period` berdasarkan *lookahead*: titik hanya menjadi bagian bilangan riil jika diikuti digit. Kasus `42.` menghasilkan `intcon + period` karena tidak ada digit setelah titik.

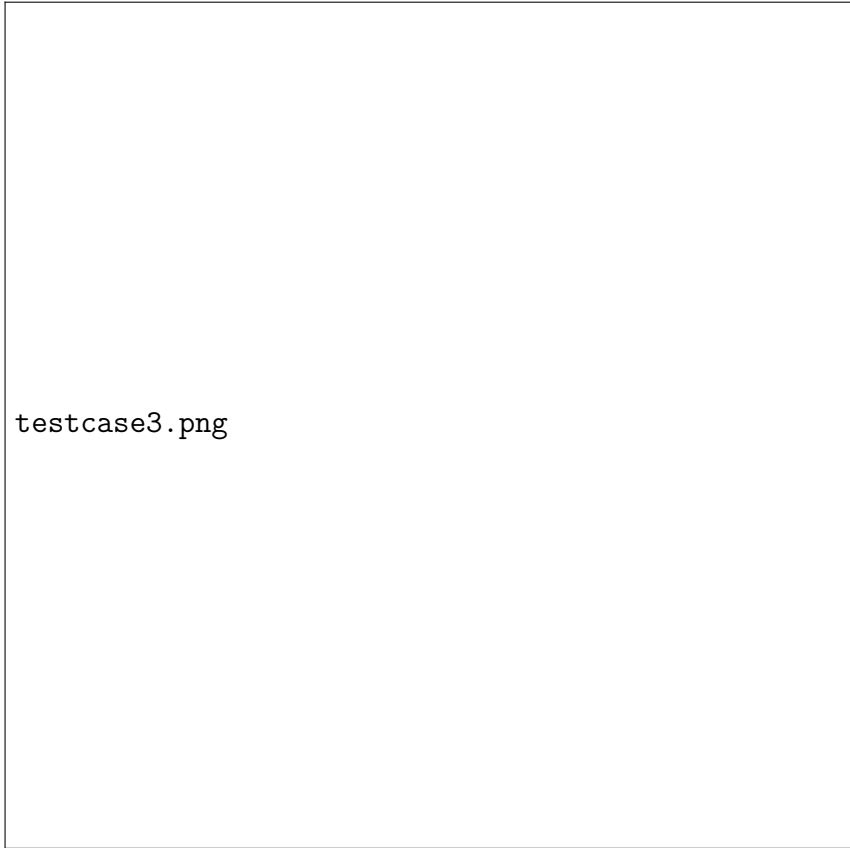
5.3 Kasus Uji 3: Charcon vs String

Berkas: `edge_charcon_vs_string.txt`

Masukan:

```
1 'a'
2 'ab'
3 'hello world'
4 'x'
5 ''
```

Keluaran yang diharapkan: `charcon (a)`, `string (ab)`, `string (hello world)`, `charcon (x)`, `string ()` — total 5 token.



testcase3.png

Gambar 6: Hasil pengujian kasus uji 3

Analisis: Literal dengan tepat satu karakter menjadi `charcon`; selainnya (termasuk literal kosong `"`) menjadi `string`. Perilaku ini sesuai dengan logika `val.length() == 1` pada `stringLiteral()`.

5.4 Kasus Uji 4: Komentar Bersarang dan Berkas Hanya Komentar

Berkas: `edge_nested_comment.txt` dan `edge_only_comments.txt`

Masukan (komentar bersarang):

```
1 { comment with { nested brace }
2 x
```

Keluaran yang diharapkan: `ident (x)` — komentar dikonsumsi seluruhnya, hanya `x` yang tersisa.

Masukan (hanya komentar): Berkas yang hanya berisi komentar `{}` dan `(* *)`.

Keluaran yang diharapkan: Tidak ada token yang dicetak.



Gambar 7: Hasil pengujian kasus uji 4

Analisis: Komentar tidak bersarang; kurung kurawal `{}` di dalam komentar diabaikan. Berkas yang hanya mengandung komentar menghasilkan keluaran kosong, membuktikan bahwa komentar tidak menghasilkan token.

5.5 Kasus Uji 5: Konstruksi yang Tidak Ditutup

Berkas: `edge_terminated.txt` dan `edge_terminated_comment.txt`

Masukan: String literal tanpa kutip penutup (`'hello'`) dan komentar tanpa kurung penutup (`{ never ends`).

Keluaran yang diharapkan (stderr):

```
1 [line 2] Error: Unterminated string or character literal.
2 [line 2] Error: Unterminated comment.
```



Gambar 8: Hasil pengujian kasus uji 5

Analisis: Lexer melaporkan kesalahan dengan nomor baris yang akurat dan tidak mengalami *crash*. Scanner meneruskan pemrosesan setelah menemukan kesalahan (*error recovery*).

5.6 Kasus Uji 6: Operator == vs Karakter = Tunggal

Berkas: edge_eq1_vs_single_eq.txt

Masukan:

```
1 x == 5
2 y = 3
3 == ==
```

Keluaran yang diharapkan (stdout): ident (x), eql, intcon (5), ident (y), intcon (3), eql, eql.

Keluaran yang diharapkan (stderr): [line 2] Error: Unexpected character:
=



Gambar 9: Hasil pengujian kasus uji 6

Analisis: Operator `==` dikenali sebagai `eq1`, sedangkan karakter `=` tunggal (tanpa diikuti `=`) menghasilkan kesalahan karena bukan operator valid dalam Arion.

5.7 Kasus Uji 7: Operator `:=` vs `:` Diikuti Karakter Lain

Berkas: `edge_becomes_vs_colon.txt`

Masukan:

```
1 x := 10
2 y : integer
3 a :=b
4 c: d
```

Keluaran yang diharapkan: `ident (x), becomes, intcon (10), ident (y), colon, ident (integer), ident (a), becomes, ident (b), ident (c), colon, ident (d)` — total 12 token.



Gambar 10: Hasil pengujian kasus uji 7

Analisis: Mekanisme `match('=')` berhasil membedakan `:=` (becomes) dari `:` (colon). Perhatikan bahwa `integer` bukan kata kunci dalam Arion sehingga dikenali sebagai `ident`.

5.8 Ringkasan Hasil Pengujian

No.	Deskripsi Kasus Uji	Status	Kesalahan
1	Kata kunci berbagai <i>case</i>	Lulus	Tidak ada
2	Bilangan riil vs titik	Lulus	Tidak ada
3	Charcon vs string	Lulus	Tidak ada
4	Komentar bersarang & hanya komentar	Lulus	Tidak ada
5	Konstruksi tidak ditutup	Lulus	2 kesalahan (sesuai harapan)
6	<code>==</code> vs <code>=</code> tunggal	Lulus	1 kesalahan (sesuai harapan)
7	<code>:=</code> vs <code>:</code>	Lulus	Tidak ada

Tabel 4: Ringkasan hasil pengujian

Seluruh kasus uji menghasilkan keluaran yang sesuai dengan ekspektasi. Pesan kesalahan pada kasus uji 5 dan 6 muncul pada `stderr` dengan nomor baris yang tepat.

6 Akhiran

6.1 Kesimpulan

Pada milestone pertama ini, kelompok berhasil merancang dan mengimplementasikan *lexical analyzer* untuk bahasa Arion dengan hasil sebagai berikut:

1. **DFA berbasis modul** — Lexer berhasil mengimplementasikan DFA dengan pendekatan modular yang memisahkan pengenalan identifier, bilangan, string, komentar, dan operator ke dalam modul-modul independen. Seluruh 52 jenis token dapat dikenali dengan benar.
2. **Penanganan ambiguitas** — Kasus-kasus ambigu seperti perbedaan `charcon` vs `string` (berdasarkan panjang), `intcon` vs `realcon` (berdasarkan *lookahead* setelah titik), dan operator satu vs dua karakter (menggunakan `match()`) berhasil ditangani secara deterministik.
3. **Error recovery** — Lexer mampu melaporkan kesalahan leksikal (karakter tidak valid, literal/komentar tidak ditutup) tanpa menghentikan proses scanning, sehingga seluruh kesalahan dalam satu berkas dapat dilaporkan sekaligus.

6.2 Tantangan yang Dihadapi

1. **DFA vs *switch-case*** — Terdapat *tradeoff* antara implementasi DFA eksplisit (tabel transisi) dan pendekatan *switch-case* yang lebih prosedural. Pendekatan *switch-case* dipilih karena lebih mudah dibaca dan di-*debug*, meskipun kurang fleksibel untuk modifikasi pola secara otomatis.
2. **Ambiguitas `charcon` vs `string`** — Kedua tipe menggunakan pembatas yang sama (`'`), sehingga klasifikasi baru dapat dilakukan setelah seluruh isi literal terkumpul. Solusinya adalah memeriksa panjang setelah penutupan kutip.
3. **Komentar tidak menghasilkan token** — Keputusan desain untuk tidak menghasilkan token `comment` menyederhanakan antarmuka dengan parser, namun memerlukan perhatian khusus agar komentar yang tidak ditutup terdeteksi sebagai kesalahan.

Daftar Pustaka

- [1] A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston: Pearson, 2007.
- [2] J. E. Hopcroft, R. Motwani, dan J. D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed. Boston: Pearson, 2006.
- [3] R. W. Sebesta, *Concepts of Programming Languages*, 12th ed. Boston: Pearson, 2019.
- [4] TutorialsPoint, "Compiler Design – Lexical Analysis," [Online]. Available: https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.htm. [Accessed: Apr. 2, 2026].

Appendix

Sumber Daya	Tautan
Repository GitHub	https://github.com/nicholaswisee/HBB-Tubes-IF2224-2026
Deployment	https://your-deployment-url.example.com
Diagram DFA	https://drive.google.com/file/d/1XT-esS_K76ImoWZUwJKy43QvJW_NdP0a/view

Tabel 5: Tautan repository, deployment, dan diagram

Pembagian Tugas

Nama	NIM	Kontribusi
Nathanael Imandatua Manurung	13524021	25%
Nicholas Wise Saragih Sumbayak	13524037	25%
Kurt Mikhael Purba	13524065	25%
Rava Khoman Tuah Saragih	13524107	25%

Tabel 6: Pembagian kontribusi anggota kelompok